Table of Contents

Coroutines **1**
Suspend Functions 1
Dispatchers.IO **3**
API Handling – GET Requests **3**
Anonymous Types **7**
LazyColumn **11**
Parsing Values of Anonymously Typed JSON Objects 13
Retrieving a List with a GET request 15
API Handling - POST Request **22**

Coroutines

Coroutines in Koltin allow for asynchronous execution of code. A coroutine is like a background task that run and wait for completion without freezing your app. The code below will not work in Kotlin UI because it blocks the UI.

```
fun fetchData() {       // Causes UI to freeze
    Thread.sleep(2000)  // Simulates delay
    println("Done!")
}
```

Coroutines can be started by several structures in Kotlin including **async** and **launch**.

Suspend Functions

Kotlin provides suspend functions as a lightweight way to modularize asynchronous operations. Suspend functions can only be called from within a coroutine or another suspend function. Suspend functions cannot be directly called from regular, non-suspending functions.

```
suspend fun fetchData() { // Does not block UI
    delay(2000)           // Simulates delay
    println("Done!")
}
```

Example 1: Calling a Suspend Function

This example demonstrates how to call a suspend function from a coroutine.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        Log.d("*** onCreate()", "Start of onCreate()")

        lifecycleScope.launch { // start coroutine
            val weather = fetchWeather()
            Log.d("*** onCreate()", weather)
        }
        Log.d("*** onCreate()", "End of onCreate()")
    }
}

suspend fun fetchWeather(): String {
    Log.d("*** fetchWeather()", "Start of delay")
    delay(2000)
    Log.d("*** fetchWeather()", "End of delay")
    return "Pat's app. It is sunny"
}
```

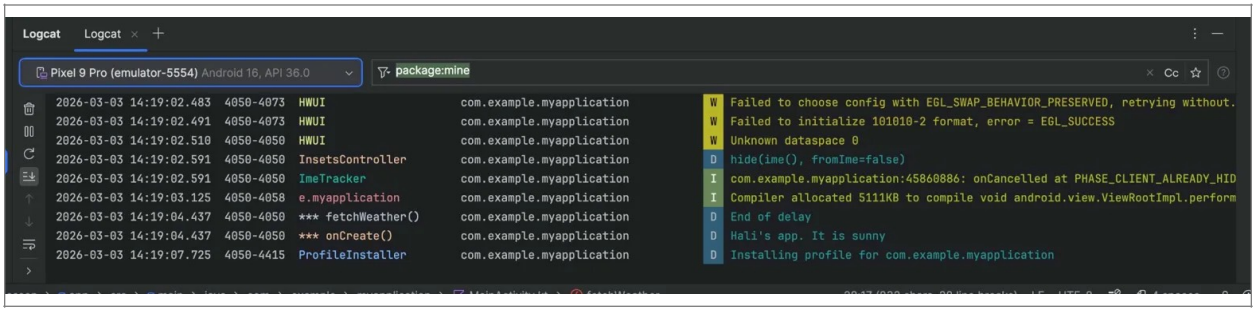
Exercise 1 (1 mark)

Why must fetchWeather() be called from a coroutine? Be specific.

Suspend functions can pause and wait (like waiting for data from the internet). Regular functions can't pause, they must run and finish immediately. Coroutines are designed to pause and resume, so only they can call suspend functions.

Exercise 2 (2 marks)

Show a screenshot of the output in Logcat from Example 1 but be sure to display your name instead of mine.



Explain the sequence of statements that are printed with Log.d().

1. First, Start of onCreate() is printed when the app starts.	2. Then the coroutine is launched in the background to run fetchWeather().
3. End of onCreate() prints immediately , it doesn't wait for the coroutine to finish.	4. Meanwhile, fetchWeather() starts and prints Start of delay, then waits 2 seconds.

5. After 2 seconds, End of delay prints, then finally Hali's app. It is sunny is returned and printed.	

Dispatchers.IO

Dispatchers are used in Kotlin for slow operations that must wait for external resources such as fetching content from the internet, reading/writing to disk, or querying a database. These operations do not need much CPU, but they take time because they are waiting for an external response.

If you run I/O tasks on the main thread (UI thread), your Kotlin app could freeze or crash. Kotlin coroutines with dispatchers let you move those tasks to a safe background thread pool that is designed for I/O work.

Example 2: Using Dispatchers

This example demonstrates how to modify Example 1 so it uses Dispatchers.IO. To build this example, replace the declaration for weather with this version.

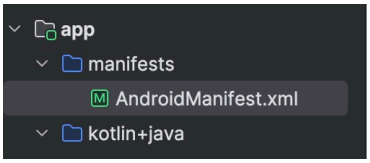
```
val weather = withContext(Dispatchers.IO) {
    fetchWeather()
}
```

API Handling – GET Requests

When accessing JSON from an external resource, the LaunchedEffect command starts the cooroutine and the suspend function encapsulates the logic for the asynchronous task.

Example 3: API Handling GET Requests

This example demonstrates how to make a GET request to obtain weather data in JSON format from https://api.open-meteo.com. First, to avoid receiving a "missing Internet permission error", add the XML uses-permisssion element to your Kotlin to the AndroidManifest.xml file.




```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools">
4      <uses-permission android:name="android.permission.INTERNET" />
5      <application
```

:

```
<uses-permission android:name="android.permission.INTERNET" />
```

It is interesting to note though that a suspend function is used to make the request so the routine will wait until the data is received before sending the data to the main application.

GetWeather.kt

```
data class WeatherInfo(val temperature: String)

suspend fun fetchWeatherInfo(): WeatherInfo {
    return withContext(Dispatchers.IO) {
        try {
            val latitude  =   49.2827
            val longitude = -123.1207

            // Use this import when prompted.
            // import java.net.URL
            val url       =
 URL("https://api.open-meteo.com/v1/forecast?latitude=$latitude&longitude=$longitude&current_weather=true")
            val connection = url.openConnection() as HttpURLConnection
            connection.requestMethod = "GET"

            if (connection.responseCode == HttpURLConnection.HTTP_OK) {
                val response = connection.inputStream.bufferedReader().use {
                    it.readText() }
                val json     = JSONObject(response)
                val current  = json.getJSONObject("current_weather")
                val temp     = current.getDouble("temperature")

                // Combine into one WeatherInfo object
                WeatherInfo(temperature = "$temp")
            } else {
                WeatherInfo("N/A")
            }
        } catch (e: Exception) {
            WeatherInfo(e.message ?: "Unknown error")
        }
    }
}
```

MainActivity.kt stores the code for the composable function which displays Vancouver's temperature. LaunchedEffect is a composable function which allows the launch of a coroutine within a composable function.

```
@Composable
fun WeatherInfoDisplay() {
    var info by remember { mutableStateOf<WeatherInfo?>(null) }

    LaunchedEffect(Unit) {
        info = fetchWeatherInfo()
    }
    Scaffold() { padding ->
        Column(
            modifier = Modifier.fillMaxSize().padding(padding),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            if (info == null) {
                CircularProgressIndicator()
            } else {
                Text(text = "Temperature: ${info!!.temperature} °C", fontSize=30.sp)
            }
        }
    }
}
```

This is the call from within SetContent {} of **MainActivity**.

```
WeatherInfoDisplay()
```

Exercise 3 (1 mark)

What is the role of the Dispatchers in Example 3?

Dispatchers.IO runs the API request in the background so the app doesn't freeze while waiting for the internet response.

Exercise 4 (1 mark)

Set a breakpoint in the function where the data is returned by the API and parsed. Show a screenshot of the data values within the debugger after it has been parsed into reader friendly name value pairs.

Exercise 5 (1 mark)

Explain why the suspend function was needed for running the code in Example 3. What does the suspend function do in this case? Why is the launch effect function needed?

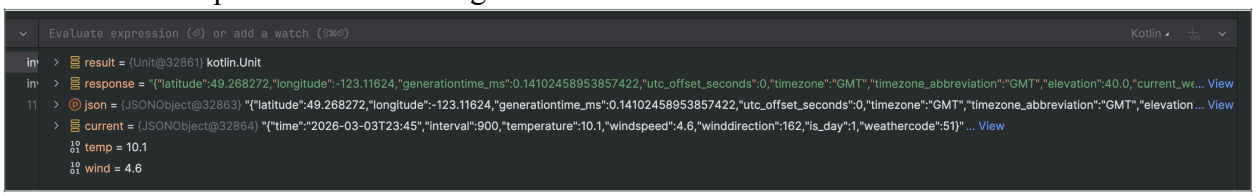
The suspend function is needed because fetching data from the internet takes time. It pauses and waits for the API response without freezing the app.
LaunchedEffect is needed because we can't call a suspend function directly inside a Composable function. LaunchedEffect starts a coroutine inside the Composable so the suspend function can be called.

Exercise 6 (1 mark)

Modify the code in Example 3 to so that it also retrieves and displays the Vancouver windspeed in addition to the Vancouver temperature. Show a screenshot of the application after it shows the windspeed and temperature for Vancouver.

Exercise 7 (1 mark)

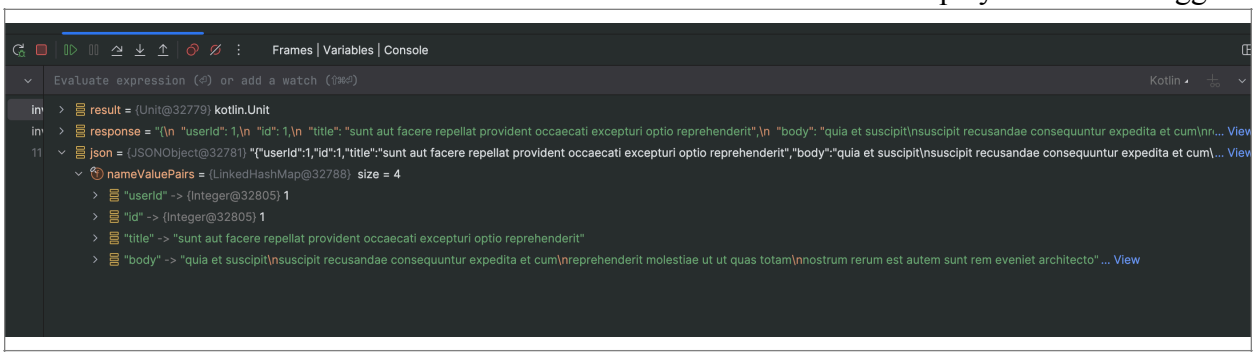
Show a screenshot of your app while the debugger is halted inside GetWeather.kt at the line after where the windspeed variable is assigned the value that is retrieved from the API.



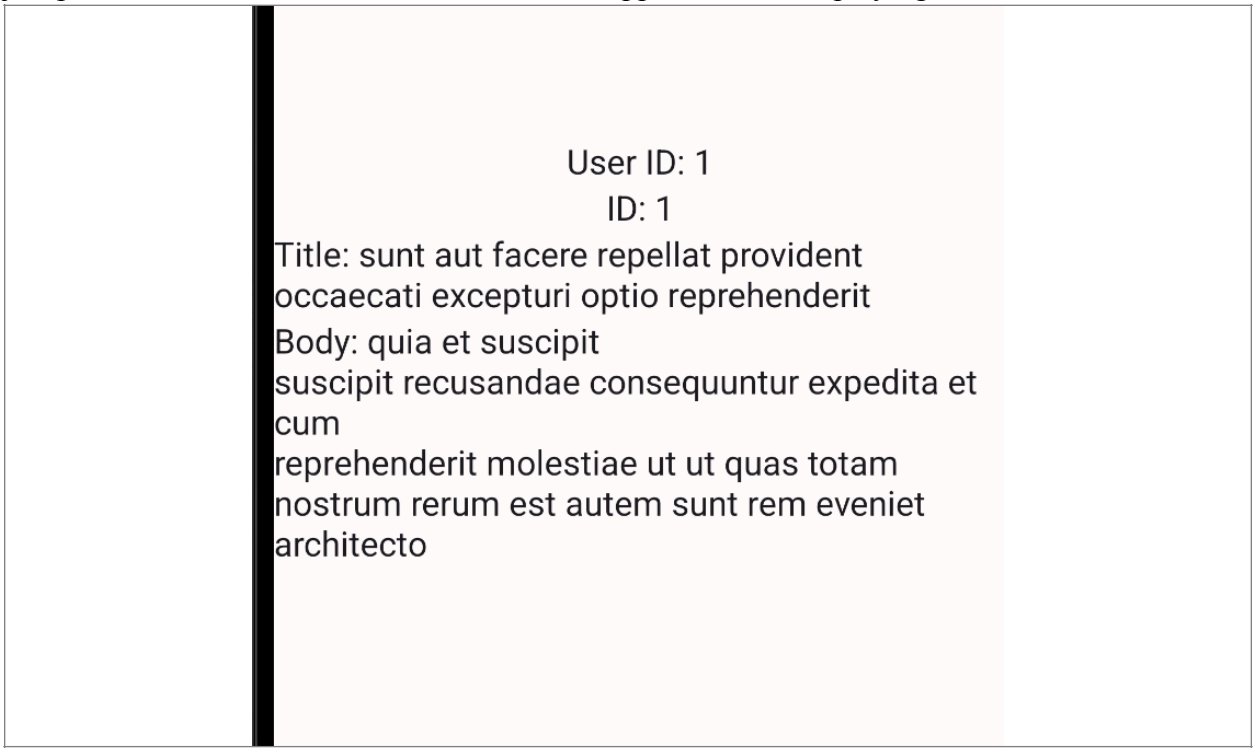
Exercise 8 (3 marks)

Modify Example 3 so it makes a get request to https://jsonplaceholder.typicode.com/posts/1"

Show a screenshot of the data that is returned from the API while it is displayed in the debugger.



Modify the app to show the values of the four name value pairs that are returned from the jsonplaceholder API. Show a screenshot of the app while it is displaying this information:

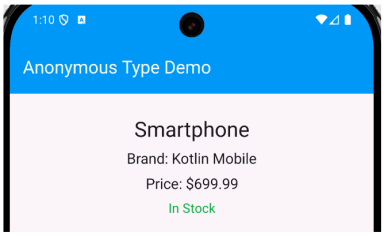


Anonymous Types

In Kotlin (and many other languages) – when working with external data sources the exact data format may be either unknown or awkward. Elegant code can always be written for efficiency and for strict type enforcement. Sometimes though it is quicker to define an anonymous type which can store varying or verbose formats without sacrificing app integrity. Anonymous types are defined with **String** and **Any?** Parameters:

```
var product by remember { mutableStateOf<Map<String, Any?>>(emptyMap())}
```

The **key** is a string. The **value** can be any type (`String`, `Int`, `Double`, `Boolean`, etc.). The mapOf() function creates a mutable assignment to the anonymous typed variable. This example below shows how to define an anonymous type and display the information from it on a phone screen.



MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { ProductScreen()}
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProductScreen() {
    // Create an anonymous-type variable
    var product by remember { mutableStateOf<Map<String, Any?>>(emptyMap()) }

    // Load fake data
    LaunchedEffect(Unit) {
        product = mapOf(
            "title" to "Smartphone",
            "brand" to "Kotlin Mobile",
            "price" to 699.99,
            "stock" to true
        )
    }

    // Phone screen layout
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Anonymous Type Demo") },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = Color(0xFF2196F3),
                    titleContentColor = Color.White
                )
            )
        }
    ) { padding ->
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            contentAlignment = Alignment.Center
        ) {
            if (product.isEmpty()) {
                CircularProgressIndicator()
            } else {
                Column(
                    horizontalAlignment = Alignment.CenterHorizontally,
                    verticalArrangement = Arrangement.spacedBy(10.dp),
                    modifier = Modifier.padding(20.dp)
```

Exercise 9 (2 marks)

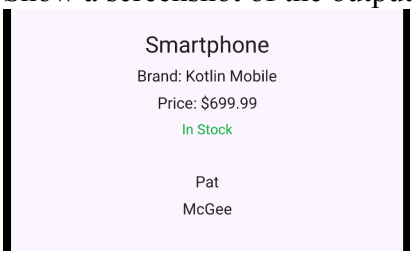
Show the code needed to create a second anonymous type variable which receives a first and last name. Also show the code which assigns the first name to the anonymous type variable and the other line which assigns the last name to the anonymous type variable.

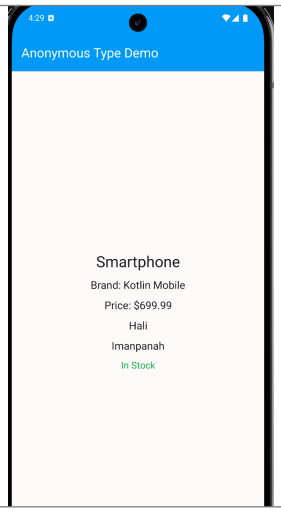
```
var person by remember { mutableStateOf<Map<String, Any?
>>(emptyMap()) }
person = mapOf(
    "firstName" to "Hali",
    "lastName" to "Imanpanah"
)
```

Show the code needed to needed to display the first and last name of your anonymous type variable on separate lines.

```
Text(text = "${person["firstName"]}", fontSize = 18.sp)
Text(text = "${person["lastName"]}", fontSize = 18.sp)
```

Show a screenshot of the output. It should look like this but with your name in it.

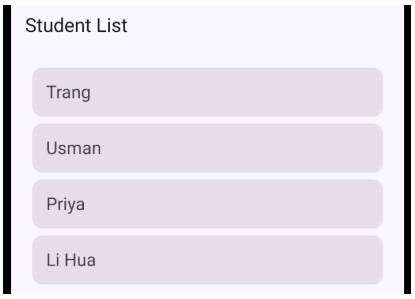




LazyColumn

Kotlin's LazyColumn was discussed last day which was two weeks ago. Since it is included later in this document, I am adding it in for a quick review. The LazyColumn is efficient because it only renders visible items and this is helpful when working with large lists. The LazyColumn also helps to make it easy to display nicely formatted list content.

Example 4: Lazy Column



MainActvity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { MyListScreen()}
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MyListScreen() {
    val names = listOf("Trang", "Usman", "Priya", "Li Hua")

    Scaffold(
        topBar = {
            TopAppBar(title = { Text("Student List") })
        }
    ) { padding ->
        // Scrollable list
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            verticalArrangement = Arrangement.spacedBy(8.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            contentPadding = PaddingValues(16.dp)
        ) {
            items(names) { name ->
                Card(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 8.dp),
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.surfaceVariant
                    )
                ) {
                    Text(
                        text = name,
                        fontSize = 20.sp,
                        modifier = Modifier.padding(16.dp)
                    )
                }
            }
        }
    }
}
```

Exercise 10 (1 mark)

What mechanism is used to iterate through all list items in Example 4?

item() is used to iterate through all list items. It automatically loops through each item in the list and displays it on screen.

Parsing Values of Anonymously Typed JSON Objects

When Anonymous types are used, the property values of the JSON object are unknown. However, a routine like the one in Example 5 can be used to parse the property values and store their values in a proper format such as String, Boolean or Double.

Example 5: Parsing Values from JSON Objects

This example does not use a Kotlin screen but it can be run at the Kotlin Playground at https://play.kotlinlang.org/

```kotlin
fun main() {
    // Simulated JSON-like data using a Kotlin map
    val studentObject = mapOf(
        "name" to "Alice",
        "age" to 21,
        "grade" to 95.5,
        "major" to "Computer Science",
        "graduated" to false
    )

    val studentMap = mutableMapOf<String, Any?>()

    // Iterate through keys as if it were a JSONObject
    for ((key, value) in studentObject) {
        val mapped: Any? = when (value) {
            null -> "null"
            is Map<*, *> -> value.toString()
            is List<*> -> value.toString()
            else -> value
        }

        studentMap[key] = mapped
    }

    println(studentMap)

    // Print each key, value, and its Kotlin data type
    println("\n--- Value Types ---")

    for (entry in studentMap.entries) {                // Step 1: access map entry
        val key = entry.key                            // Step 2: get the key
        val value = entry.value                        // Step 3: get the value

        // Step 4: figure out the type name safely
        val typeName = if (value != null) {
            value::class.simpleName
        } else {
            "null"
        }
        // Step 5: print the information
        println("$key → $value  (Type: $typeName)")
    }
}
```

Exercise 11 (1 mark)

Show a screenshot of the output after running the code in Example 5 at Kotlin Playground.

```
{name=Alice, age=21, grade=95.5, major=Computer Science, graduated=false}

--- Value Types ---
name → Alice  (Type: String)
age → 21  (Type: Int)
grade → 95.5  (Type: Double)
major → Computer Science  (Type: String)
graduated → false  (Type: Boolean)
```

Exercise 12 (1 mark)

What does the Any? type represent in Kotlin?

It can hold any type of value — String, Int, Double, Boolean, or even null. It's the most flexible type in Kotlin.

Exercise 13 (1 mark)

In Example 5, why is studentMap needed if studentObject already contains the data?

because it explicitly casts the data to Map<String, Any?> so we can iterate through the key-value pairs using a for loop.

Exercise 14 (1 mark)

What does it::class.simpleName return in the second loop?

It returns the type name of the value, like "String", "Int", "Double", or "Boolean".

Exercise 15 (1 mark)

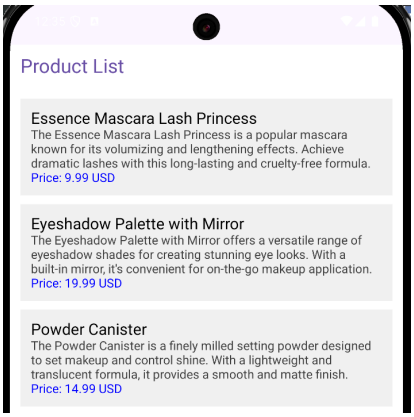
What would the output look like if "grade" were changed from 95.5 to "A+"?

grade would show as A+ with type String instead of 95.5 with type Double.

Retrieving a List with a GET request

Example 6: Showing a List

This example helps to combine content on API GET requests, anonymous types, managing JSON objects, and LazyColumns to create an app which obtains and displays a list of products that are retrieved using an API.



GetWeather.kt

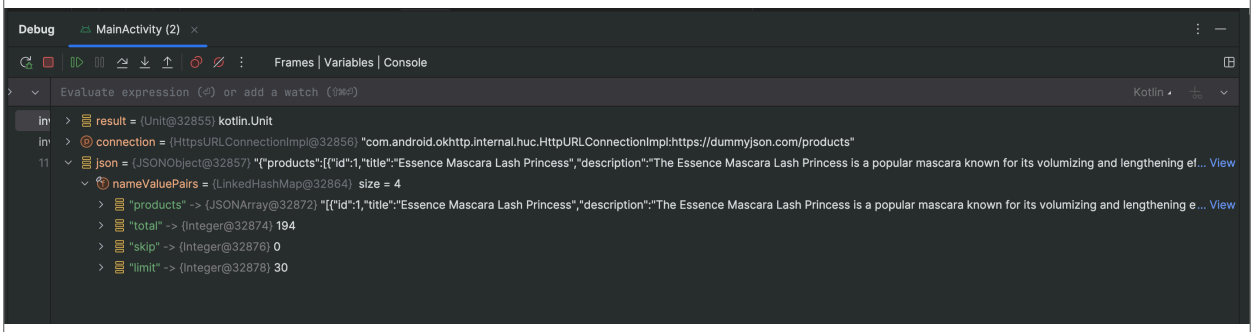
```
suspend fun fetchProducts(): List<Map<String, Any?>> = withContext(Dispatchers.IO) {
    var connection: HttpURLConnection? = null
    try {
        connection
        = (URL("https://dummyjson.com/products").openConnection() as
           HttpURLConnection).apply {
            requestMethod = "GET"
        }

        if (connection.responseCode == HttpURLConnection.HTTP_OK) {
            val response
            = connection.inputStream.bufferedReader().use { it.readText() }
            val json = JSONObject(response)
            val products = json.getJSONArray("products")

            // Convert each JSONObject in the array into a Map<String, Any?>
            val resultList = mutableListOf<Map<String, Any?>>()
            for (i in 0 until products.length()) {
                val productObject = products.getJSONObject(i)
                val productMap = mutableMapOf<String, Any?>()

                val keys = productObject.keys()
                while (keys.hasNext()) {
                    val key = keys.next()
                    val raw = productObject.opt(key) // get value for that key
                    val mapped: Any? = when (raw) {
                        JSONObject.NULL -> null

                        is JSONObject, is JSONArray -> raw.toString()
                        else -> raw
                    }
                    productMap[key] = mapped
                }

                resultList.add(productMap)
            }

            resultList
        } else {
            emptyList()
        }
    } catch (e: Exception) {
        Log.e("fetchWeatherInfo", "Error fetching products", e)
        emptyList()
    } finally {
        connection?.disconnect()
    }
```

MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { WeatherInfoDisplay()}
    }
}

@Composable
fun WeatherInfoDisplay() {
    // Nullable means "still loading"
    var products by remember { mutableStateOf<List<Map<String, Any?>>?>(null) }
    var error by remember { mutableStateOf<String?>(null) }

    LaunchedEffect(Unit) {
        try {
            products = fetchProducts() // Fetch list of product maps
        } catch (e: Exception) {
            error = e.message
            products = emptyList()
        }
    }

    Scaffold { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
                .background(Color.White)
        ) {
            when {
                error != null -> {
                    Text(
                        text = "Error: $error",
                        color = Color.Red,
                        modifier = Modifier.padding(16.dp)
                    )
                }

                products == null -> {
                    Box(
                        modifier = Modifier.fillMaxSize(),
                        contentAlignment = Alignment.Center
                    ) {
                        CircularProgressIndicator()
                    }
                }
```

Exercise 16 (2 marks)

At the parent level, when the parsed JSON object, named *json*, is first obtained after the API response, what four name proprieties exist? Hint: Check it with the debugger.



- products
- total
- skip
- limit

How is the list of products extracted from *json*?

using json.getJSONArray(“products") which gets the array named "products" from the JSON object.

Aside from the name of the function, explain the main difference between the function which creates the parsed *json* object from the response:

```
val json = JSONObject(response)
```

and the function which creates a list of products from the *json* object:

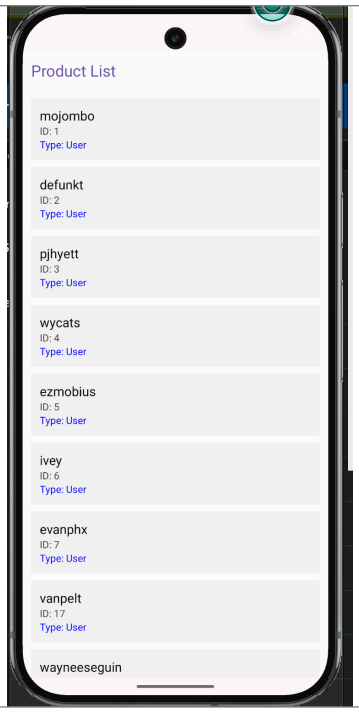
JSONObject(response) converts the raw text response into a single JSON object with key-value pairs.
json.getJSONArray("products") extracts an array (list) of multiple items from that JSON object. The main difference is that JSONObject holds one object while JSONArray holds a list of objects.

Exercise 17 (6 marks)

Modify the code in Example 6 to retrieve and display users from Github. Below is a code hint to help you get started. Choose three or four relevant user properties to display.

```
val url = URL("https://api.github.com/users")
    val connection = url.openConnection() as HttpURLConnection
    connection.requestMethod = "GET"

    try {
        if (connection.responseCode == HttpURLConnection.HTTP_OK) {
            val response = connection.inputStream.bufferedReader().use
                           { it.readText() }
            val jsonArray = JSONArray(response)
            val resultList = mutableListOf<Map<String, Any?>>()

            // Loop through each object in the array
            for (i in 0 until jsonArray.length()) {
                val userObject = jsonArray.getJSONObject(i)
                val userMap = mutableMapOf<String, Any?>()
```

Show a screenshot of your output here:



Exercise 18 (1 mark)

In Example 6, an array of products is created with the following code:

```
val json = JSONObject(response)
val products = json.getJSONArray("products")
```

In Exercise 17, the JSON array can be extracted directly from the response.

```
val jsonArray = JSONArray(response)
```

Why is the extra step needed to create the array of items in Example 6?

In Example 6, the response is a JSON object {} that contains the array inside it, so we need an extra step to get to it. In Exercise 17, the response is already a JSON array [] so we can use it directly.

API Handling - POST Request

Example 7: Making a POST request

```
suspend fun postToDummyJson(): String = withContext(Dispatchers.IO) {
    val data = mapOf(
        "title" to "Pat's new Kotlin product",
        "price" to 499,
        "category" to "Tech"
    )

    // Convert map to JSON text
    val jsonBody = data.entries.joinToString(prefix = "{", postfix = "}") {
        "\"${it.key}\": \"${it.value}\""
    }

    val url = URL("https://dummyjson.com/products/add")
    val connection = url.openConnection() as HttpURLConnection

    try {
        connection.requestMethod = "POST"
        connection.setRequestProperty("Content-Type", "application/json;
charset=utf-8")
        connection.setRequestProperty("Accept", "application/json")
        connection.doOutput = true

        // Write JSON body
        connection.outputStream.use { os ->
            os.write(jsonBody.toByteArray(Charsets.UTF_8))
        }

        // Read response
        val status = connection.responseCode
        val stream = if (status in 200..299) connection.inputStream else
connection.errorStream
        val response = stream.bufferedReader().use { it.readText() }

        "Response Code: $status\n\n$response"
    } catch (e: Exception) {
        "Error: ${e.message}"
    } finally {
        connection.disconnect()
    }
}
```

MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { DummyJsonPostScreen() }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DummyJsonPostScreen() {
    var responseText by remember { mutableStateOf("No response yet...") }
    val scope = rememberCoroutineScope()

    Scaffold(
        topBar = { TopAppBar(title = { Text("DummyJSON POST Example") }) }
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
                .padding(16.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Button(
                onClick = {
                    scope.launch {
                        responseText = "Sending request..."
                        responseText = postToDummyJson()
                    }
                }
            ) {
                Text("Send POST Request")
            }

            Spacer(modifier = Modifier.height(24.dp))

            Text(
                text = responseText,
                modifier = Modifier.padding(8.dp)
            )
        }
    }
}
```